



UNIVERSIDAD  
**NACIONAL**  
DE COLOMBIA

---

**Departamento de Ingeniería Mecánica y Mecatrónica**

**IA y Minirobots**

## **Taller 4. Programación genética**

**Autores:**

Yeison Esteban Ortega Chacua

Daniel Alberto Rodriguez Porras

Daniel Santiago Navarro Gil

**Docente:**

José Martínez

Bogotá D.C, Colombia

## Desarrollo

### 1. Decodificador de 7 segmentos

#### a. Planteamiento

El objetivo consiste en encontrar una expresión lógica independiente para cada segmento del display (a, b, c, d, e, f y g), utilizando como entradas los cuatro bits BCD A, B, C y D. Los terminales del árbol son dichas entradas y las funciones permitidas son AND, OR y NOT. La aptitud corresponde al número de casos correctos sobre los diez dígitos decimales, por lo que el valor máximo posible es 10.

| A | B | C | D | a | b | c | d | e | f | g |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 |
| 0 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 1 | 0 | 1 | 1 | 0 | 1 |
| 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 1 |
| 0 | 1 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 1 | 1 | 0 | 1 | 1 | 0 | 1 | 1 |
| 0 | 1 | 1 | 0 | 1 | 0 | 1 | 1 | 1 | 1 | 1 |
| 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 1 |

Tabla 1. Tabla de verdad del display de 7 segmentos

La función de aptitud corresponde al número de casos correctamente reproducidos entre los diez dígitos decimales:

$$\text{Aptitud} = \sum I(\hat{y}_i = y_i), \quad i = 0, \dots, 9$$

$$0 \leq \text{Aptitud} \leq 10$$

Una aptitud de 10 indica que la expresión reproduce correctamente todos los casos del segmento. La evolución se ejecutó con una población de 100 individuos, selección por torneo de tamaño 3, probabilidad de cruce de 0.7, probabilidad de mutación de 0.01 y un máximo de 50 generaciones. El mejor individuo de cada segmento se conserva mediante un HallOfFame. Además, el crecimiento de los árboles se limitó a una altura máxima de 8 para evitar estructuras excesivamente grandes.

#### b. Resultados

```
Segmento a
Expresión: OR(NOT(NOT(D)), OR(C, NOT(B)))
Aptitud: 9.0 / 10
-----
Segmento b
Expresión: OR(AND(NOT(A), AND(C, D)), NOT(AND(B, C)))
Aptitud: 9.0 / 10
-----
Segmento c
Expresión: OR(OR(D, NOT(C)), B)
Aptitud: 10.0 / 10
-----
Segmento d
Expresión: OR(NOT(AND(C, D)), AND(AND(C, C), NOT(B)))
Aptitud: 8.0 / 10
-----
Segmento e
Expresión: NOT(D)
Aptitud: 9.0 / 10
-----
Segmento f
Expresión: NOT(C)
Aptitud: 8.0 / 10
-----
Segmento g
Expresión: OR(OR(C, OR(A, A)), B)
Aptitud: 9.0 / 10
-----
```

Figura 1. Evolución final de los 7 segmentos.

Al combinar las siete expresiones, 5 de los 10 dígitos fueron representados completamente de forma correcta. Esto muestra una diferencia importante entre optimizar cada salida por separado y obtener un decodificador completo: aunque la mayoría de los segmentos alcanzan 9/10 y uno alcanza 10/10, un solo error en cualquier segmento hace que el dígito completo sea considerado incorrecto.

| Número                                    | Esperado              | Calculado             | Correcto |
|-------------------------------------------|-----------------------|-----------------------|----------|
| 0                                         | (1, 1, 1, 1, 1, 1, 0) | (0, 1, 1, 1, 1, 0, 0) | False    |
| 1                                         | (0, 1, 1, 0, 0, 0, 0) | (0, 1, 1, 0, 0, 0, 0) | True     |
| 2                                         | (1, 1, 0, 1, 1, 0, 1) | (1, 1, 0, 1, 1, 0, 1) | True     |
| 3                                         | (1, 1, 1, 1, 0, 0, 1) | (1, 1, 1, 1, 0, 0, 1) | True     |
| 4                                         | (0, 1, 1, 0, 0, 1, 1) | (0, 0, 1, 1, 1, 1, 1) | False    |
| 5                                         | (1, 0, 1, 1, 0, 1, 1) | (1, 0, 1, 1, 0, 1, 1) | True     |
| 6                                         | (1, 0, 1, 1, 1, 1, 1) | (1, 0, 1, 1, 1, 1, 1) | True     |
| 7                                         | (1, 1, 1, 0, 0, 0, 0) | (1, 1, 1, 1, 0, 1, 1) | False    |
| 8                                         | (1, 1, 1, 1, 1, 1, 1) | (1, 1, 1, 1, 1, 1, 1) | True     |
| 9                                         | (1, 1, 1, 1, 0, 1, 1) | (1, 1, 1, 1, 0, 1, 1) | True     |
| Dígitos correctamente representados: 7/10 |                       |                       |          |

Tabla 2. Verificación de soluciones.

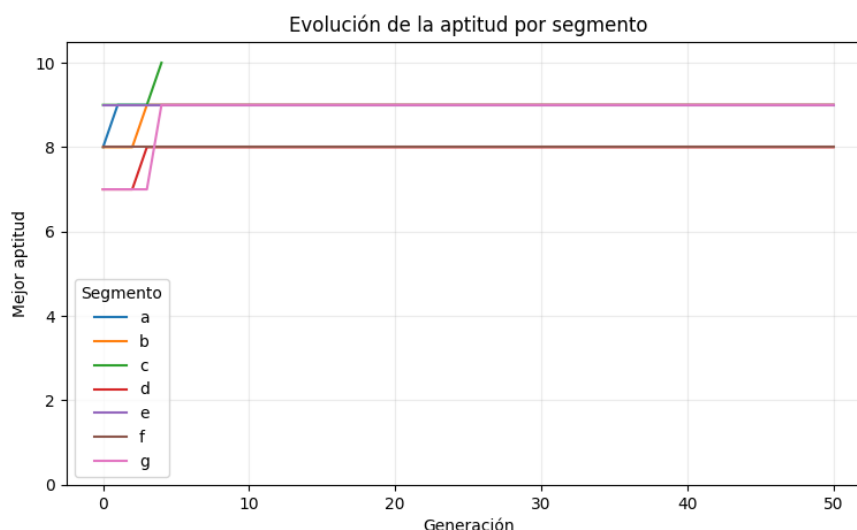


Figura 2. Evolución de la aptitud por segmento.

La gráfica muestra que la mayoría de los segmentos encuentra rápidamente soluciones de alta aptitud, y las mejoras se presentan antes de la décima generación, por lo que concluimos que para este problema no se requieren tantas iteraciones.

Para más información, visite el gitHub [IA-y-Minirobots](#) [2], y descargue el Notebook de Google Colab [Ejercicio 1 Programacion Genetica 7 segmentos](#), donde se expone y explica el código de manera más explícita.

## 2. Detección de Fraudes

### a. Planteamiento

Se generaron 300 transacciones artificiales descritas por monto, hora del día, distancia a la residencia e ingreso mensual. La PG busca una regla que clasifique cada transacción como fraude o no fraude mediante operaciones aritméticas, comparaciones y operadores lógicos.

Para hacer los datos artificiales más razonables, se considera sospechosa una transacción si presenta un monto alto respecto al ingreso, ocurre de madrugada o se realiza lejos de la residencia. Se marca fraude cuando se cumplen al menos dos condiciones. Estas reglas solo generan los datos; la PG deberá descubrir una regla a partir de ellos.

| Grupo       | Elementos                                              | Tipo de salida |
|-------------|--------------------------------------------------------|----------------|
| Terminales  | Monto, Hora, Distancia, Ingreso, constantes aleatorias | float          |
| Aritméticas | add, sub                                               | float          |
| Comparación | mayor, menor                                           | bool           |
| Lógicas     | AND, OR                                                | bool           |

Se contabilizan verdaderos positivos (TP), verdaderos negativos (TN), falsos positivos (FP) y falsos negativos (FN), y se calculan:

$$\text{Sensibilidad} = \text{TP} / (\text{TP} + \text{FN})$$

$$\text{Especificidad} = \text{TN} / (\text{TN} + \text{FP})$$

$$\text{Aptitud} = (\text{Sensibilidad} + \text{Especificidad}) / 2$$

Esta elección asigna el mismo peso a la capacidad de detectar fraudes y a la capacidad de reconocer transacciones legítimas, evitando que una clase con mayor frecuencia domine por sí sola el valor de aptitud.

|   | Monto        | Hora | Distancia | Ingreso      | Fraude |
|---|--------------|------|-----------|--------------|--------|
| 0 | 4.678218e+06 | 20   | 60.730462 | 3.352099e+06 | 1      |
| 1 | 1.076693e+06 | 23   | 12.570232 | 7.602439e+06 | 0      |
| 2 | 1.394582e+06 | 12   | 36.181425 | 2.237115e+06 | 0      |
| 3 | 6.484331e+05 | 17   | 95.295411 | 7.967537e+06 | 0      |
| 4 | 2.244360e+06 | 0    | 57.866909 | 4.812087e+06 | 1      |
| 5 | 1.504871e+06 | 2    | 90.846396 | 3.367640e+06 | 1      |
| 6 | 3.081819e+06 | 16   | 58.825840 | 8.987801e+06 | 0      |
| 7 | 1.909268e+06 | 18   | 47.695460 | 4.718767e+06 | 0      |
| 8 | 7.525603e+05 | 15   | 10.012940 | 6.538157e+06 | 0      |
| 9 | 3.608124e+06 | 14   | 74.536898 | 4.211467e+06 | 1      |

Figura 3. Ejemplo de transacciones generadas.

La evolución empleó 100 individuos, selección por torneo de tamaño 3, probabilidad de cruce de 0.7, probabilidad de mutación de 0.01 y 50 generaciones. El mejor individuo se conservó mediante un HallOfFame y se limitó la altura de los árboles a 8.

## b. Resultados

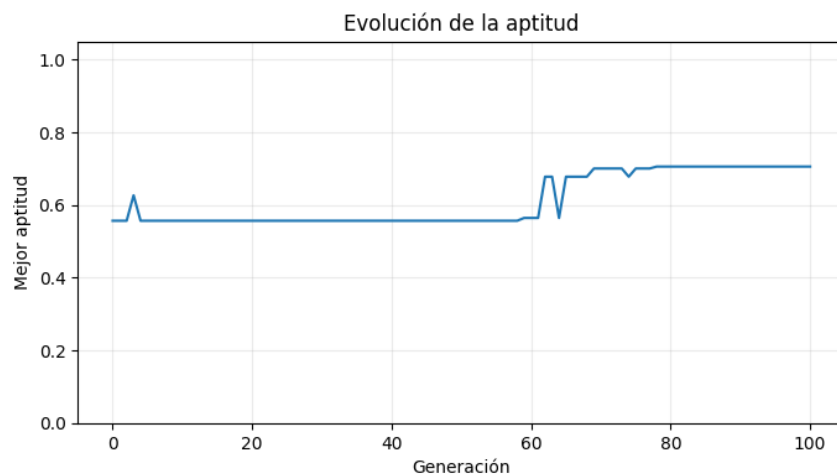


Figura 4. Evolución de la aptitud.

Como limitación, la aptitud se evaluó sobre el mismo conjunto de 300 transacciones utilizado durante la evolución. En consecuencia, el resultado demuestra el ajuste sobre los datos generados, pero no permite afirmar directamente el mismo desempeño ante transacciones nuevas. Una mejora futura sería separar datos de entrenamiento y prueba, o repetir varias corridas independientes para estimar la variabilidad del método.

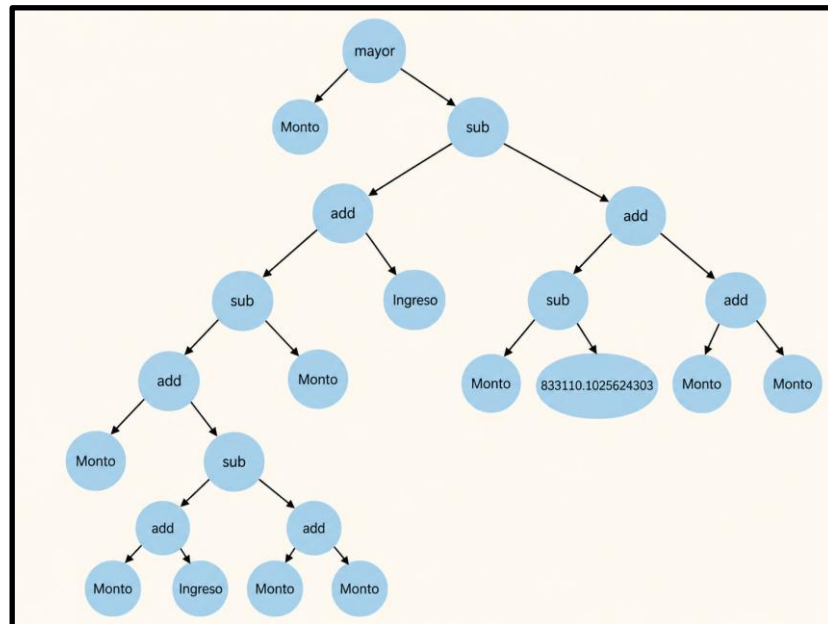


Figura 5. Árbol de la mejor expresión encontrada.

## Referencias

- [1] J. J. Martínez P., Autómatas celulares, material de clase, sept. 2026.
- [2] Rodríguez, D. (2026). IA-y-Minirobots [Repositorio de código]. GitHub.
- [3] J. J. Martínez P., Programación genética: conceptos básicos, material de clase, sept. 2026.
- [4] F.-A. Fortin et al., "DEAP: Evolutionary Algorithms Made Easy," Journal of Machine Learning Research, vol. 13, pp. 2171–2175, 2012.

**Nota:** En la elaboración de este documento se utilizó ChatGPT como herramienta de apoyo para la organización, redacción y revisión del contenido. Las fuentes citadas fueron consultadas y verificadas por los autores